VIVA PRESENTATION & TECHNICAL DEFENSE GUIDE

AI CyberShield: Intelligent Phishing Detection Platform

A full-stack, machine-learning-driven cybersecurity system combining automated 35-feature URL analysis, multi-layered heuristics (SSL, DNS, WHOIS), email header & link parsing, and real-time threat intelligence.

DOMAIN	ML ACCURACY	BACKEND TECH	FRONTEND TECH
AI & Cyber Security	99.58% (PhiUSIIL Dataset)	Python FastAPI + SQLAlchemy	Next.js 14 + React + TypeScript

1 What is this Project? (Explain in 30 Seconds)

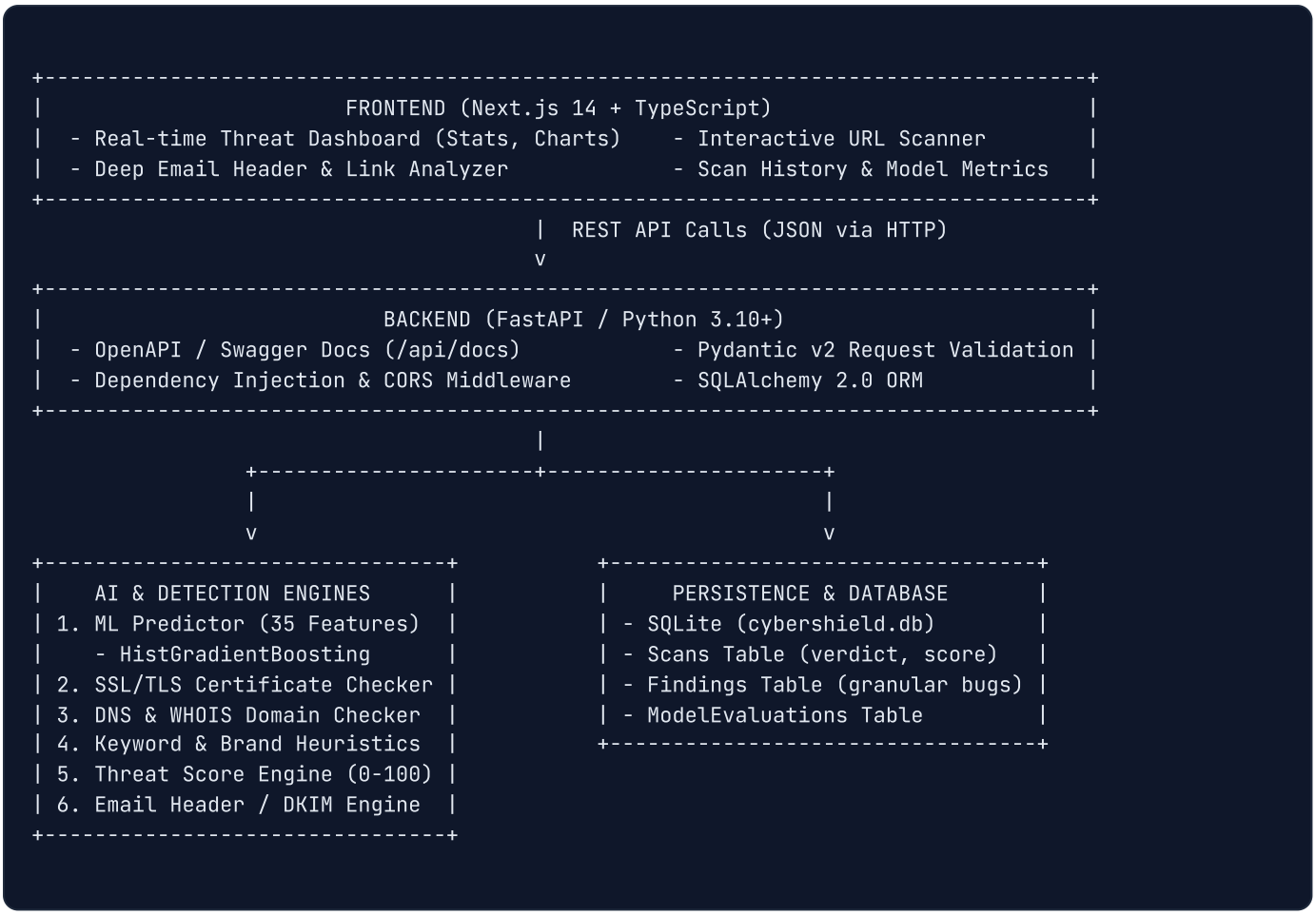
The Core Problem: Phishing is the #1 cyber attack vector in the world. Traditional antivirus and firewalls use "blacklists" (lists of known bad websites). But hackers create thousands of brand-new phishing websites every single minute. A blacklist cannot stop a website created 5 minutes ago.

Our Solution: AI CyberShield uses **Machine Learning (Artificial Intelligence)** and **Multi-Layered Security Verification** to detect phishing websites in *real-time* — even if the website has never been seen before in history.

🎯 **EXACT WORDS TO SAY TO THE EXTERNAL EXAMINER:**

"Sir/Madam, traditional security relies on static blacklists which fail against newly registered domains (zero-day phishing). Our project, AI CyberShield, solves this by analyzing the structural patterns of the URL using a trained Machine Learning model together with real-time checks on SSL certificates, domain age, DNS records, and email headers to calculate an automated threat score from 0 to 100 within milliseconds."

2 Complete System Architecture & Technologies Used



Summary of All Technologies Used:

Layer	Technology	Purpose / Role in the Project
Frontend	Next.js 14, React, TypeScript	Fast, responsive Single-Page Dashboard with client-side state and high-performance routing.
UI Styling	Modern CSS & Lucide Icons	Glassmorphic dark-theme interface with risk gauges, status rings, and badge components.
Backend API	Python 3.10+, FastAPI, Uvicorn	Asynchronous high-throughput REST API with automatic OpenAPI (Swagger) documentation.
Machine Learning	Scikit-Learn, NumPy, Pandas	Feature extraction (35 lexical indicators) and HistGradientBoosting Classifier for inference.
Database / ORM	SQLite, SQLAlchemy 2.0	Stores scan history, granular threat findings, and offline model validation metrics.
Security Libraries	dnspython, python-whois, cryptography	Performs live DNS resolution, WHOIS domain creation queries, and SSL certificate inspection.

3 How the Website Works (Step-by-Step Flow)

Step 1	Step 2	Step 3
INPUT SUBMISSION	PARALLEL SCANNING	VERDICT & FINDINGS
User enters a website URL or pastes email text/headers.	Backend triggers ML extraction + SSL + DNS + WHOIS checks.	Threat Score (0-100) calculated, stored in DB, rendered live.

Detailed Lifecycle of a Scan:

1. **Client Request:** The user types a URL (e.g., `http://secure-login-paypal.com.account-update.xyz/verify`) into the Next.js frontend and clicks *Analyze URL*.
2. **API Validation:** The request hits `POST /api/scan/url` on FastAPI. Pydantic validates the URL schema.
3. **Layer 1 — ML Feature Extraction (35 Features):** The system extracts structural features from the raw string without needing to visit or execute malicious code.
   - URL length, path length, query length, character entropy (randomness).
   - Count of dots, hyphens, slashes, '@' symbols, digits, and suspicious TLDs (e.g., `.xyz`, `.top`).
   - Subdomain count (phishing sites often stack subdomains to trick mobile users).
4. **Layer 2 — ML Classification:** The feature vector is passed to our pre-trained `HistGradientBoostingClassifier`, which outputs a Phishing Probability (e.g., 98.4%).
5. **Layer 3 — SSL/TLS Inspection:**
   - Is the certificate valid, expired, or self-signed?
   - Does the Common Name (CN) match the domain?
   - Is HTTP plain text used instead of HTTPS?
6. **Layer 4 — WHOIS & Domain Age Check:**
   - Phishing sites are typically registered only a few days before an attack. If domain age < 30 days, a severe risk flag is raised.
7. **Layer 5 — DNS & Keyword Heuristics:**
   - Resolves A/AAAA records. Checks for brand spoofing (e.g., using "paypal" or "netflix" inside an untrusted domain name).
8. **Threat Score Aggregator:** All signals are combined into an overall score (0 to 100):
   - **0 – 24:** LOW RISK (Safe / Legitimate)
   - **25 – 49:** MEDIUM RISK (Suspicious elements present)
   - **50 – 74:** HIGH RISK (Probable Phishing threat)
   - **75 – 100:** CRITICAL RISK (Confirmed Dangerous Attack)
9. **Database & UI Display:** The result is saved in SQLite and returned as JSON to the frontend. The UI animates a score gauge, displays actionable warnings, and logs the scan in the audit history.

4 The Machine Learning Pipeline (Everything in Simple Words)

Dataset: PhiUSIIL Phishing URL Dataset

- **Total Records:** 235,795 verified URLs
- **Training Set:** 188,636 URLs (80%)
- **Testing Set:** 47,159 URLs (20%)
- **Classes:** Balanced split of Legitimate and Phishing websites.

Model Performance Metrics

- **Accuracy:** 99.58%
- **Precision:** 99.91% (Almost zero false accusations)
- **Recall:** 99.11% (Catches 99 out of 100 phishing sites)
- **ROC-AUC:** 0.9983 (Near-perfect discrimination)
- **False Positive Rate:** 0.07%

🎯 **EXAMINER MIGHT ASK: "WHY DID YOU CHOOSE HISTGRADIENTBOOSTING OVER SIMPLE RANDOM FOREST OR LOGISTIC REGRESSION?"**

YOUR ANSWER:

"Sir, HistGradientBoosting groups continuous numerical features into integer-valued bins (similar to LightGBM). This makes training and prediction over 10x faster with 235,000 samples, drastically reduces memory usage, naturally handles missing values, and gave us a higher accuracy of 99.58% compared to traditional models."

5 Email Scanner & Header Analysis Engine

> Phishing attacks typically arrive through fraudulent emails. AI CyberShield includes an automated Email Inspector that analyzes three critical dimensions:

Verification Area	What It Checks	Why It Matters in Phishing
SPF (Sender Policy Framework)	Validates if the sending mail server IP is authorized by the sender's domain DNS.	Prevents basic email spoofing (e.g., someone pretending to send mail from `ceo@google.com`).
DKIM (DomainKeys Identified Mail)	Verifies cryptographic digital signature attached to email headers.	Ensures the email body was not intercepted or altered in transit.
DMARC Alignment	Checks domain policy enforcing action if SPF or DKIM fails.	Tells the receiving server whether to reject or quarantine fake emails.
Urgency & Panic Words	Scans body text for psychological triggers: *"Account suspended", "Action required in 24 hours", "Verify bank PIN"*.	Phishers rely on artificial panic to force users into making hasty mistakes.
Embedded Link Extraction	Extracts every clickable hyperlink inside the email HTML/text.	Pipes each extracted URL into our 35-feature ML Engine to find hidden traps.

6 Top 15 Viva Questions & Winning Answers

Q1. What is the motivation behind building AI CyberShield?

Answer: Traditional defense systems use signature or blacklist matching. Hackers bypass this easily by spinning up short-lived domains using bulletproof hosting. AI CyberShield uses machine learning to detect phishing by analyzing intrinsic behavioral and structural patterns of URLs and emails rather than relying on prior blacklists.

Q2. What features does your ML model extract from a URL?

Answer: It extracts 35 structural and lexical features including: total URL length, domain length, path depth, character entropy (randomness), count of special symbols (`@`, `?`, `-`, `%`, `=`), IP address presence, count of subdomains, and suspicious Top-Level Domains (TLDs).

Q3. Why is URL Entropy an important feature?

Answer: Entropy measures the randomness of characters in a string. Legitimate websites use readable dictionary words (e.g., `amazon.com/orders`). Phishing URLs generated by automated malware kits often contain long pseudo-random hashes or obfuscated strings (high entropy), making entropy a strong indicator of malicious intent.

Q4. Does HTTPS guarantee that a website is safe?

Answer: No! Many people mistakenly think a padlock icon means safe. Today, over 80% of phishing sites use free SSL certificates (like Let's Encrypt). HTTPS only encrypts the connection between the user and the server; it does not verify whether the server owner is honest or a cyber criminal. Our system checks certificate age, issuer, and domain matching.

Q5. What is the difference between Precision and Recall in this project?

Answer: Precision (99.91%): Out of all sites our model called "phishing", 99.91% were truly phishing (extremely low false alarms).
Recall (99.11%): Out of all actual phishing sites out there, our model successfully detected 99.11% of them. In cybersecurity, high recall is vital so zero threats slip through.

Q6. How is the Overall Threat Score (0–100) calculated?

Answer: It is a weighted hybrid calculation. The Machine Learning prediction contributes 60% of the weight, SSL/TLS checks contribute 15%, WHOIS domain age contributes 15%, and DNS/Keyword heuristics contribute 10%. If any critical failure is detected (such as an active brand spoof or expired SSL), extra penalty points are added.

Q7. What happens if WHOIS or DNS lookup times out? Does the whole scan crash?

Answer: No, we implemented isolated fault-tolerant try-except blocks for each engine layer. If WHOIS times out, the backend logs a partial warning, gracefully falls back, and computes the verdict using the ML model and available signals without crashing the user interface.

Q8. What is the role of SQLite and SQLAlchemy in this project?

Answer: SQLite provides lightweight, zero-configuration local relational storage for scan history, threats, and offline model evaluation metrics. SQLAlchemy provides the Object Relational Mapper (ORM) to query and write data using structured Python classes instead of writing raw SQL strings.

Q9. Why did you build the frontend in Next.js instead of simple HTML/CSS or basic React?

Answer: Next.js offers modern component architecture, TypeScript type safety, automated code-splitting, fast client-side navigation, and seamless state management. It provides a production-grade dashboard experience with modular reusable components for scan cards, gauges, and tables.

Q10. What are Homograph / Punycode Attacks?

Answer: Phishers substitute Latin letters with identical-looking Cyrillic or Greek characters (e.g., replacing 'a' with Cyrillic 'a'). When registered, the browser translates it into Punycode (e.g., `xn--...`). Our keyword and URL preprocessor checks for punycode encoding and flags homograph brand impersonations.

Q11. What is the role of Pydantic in FastAPI?

Answer: Pydantic handles request and response data validation. When a request is sent, Pydantic automatically checks data types (ensuring URLs are valid strings, numbers are within ranges) and generates automatic Swagger/OpenAPI documentation schemas.

Q12. How did you handle CORS (Cross-Origin Resource Sharing)?

Answer: Since our Next.js frontend runs on port 3000 and FastAPI runs on port 8000, browser security blocks cross-origin requests by default. We added FastAPI's `CORSMiddleware` to explicitly authorize cross-origin API calls between the two origins.

Q13. What are the limitations of this project?

Answer: 1) If an attacker compromises a legitimate, aged domain (like a hacked school or corporate server), the WHOIS age check will look safe, so the system must rely purely on ML lexical cues. 2) Websites requiring two-factor or login walls cannot have their full internal pages analyzed without authenticated crawling.

Q14. What are future enhancements for this project?

Answer: 1) Integrating a headless browser sandbox (like Playwright) to capture visual screenshots and run Computer Vision models against login form clones. 2) Deploying a lightweight Chrome/Firefox browser extension that intercepts URLs in real-time before page load.

Q15. Can you summarize your project in three bullet points?

Answer: 1. **Predictive:** Uses Machine Learning trained on 235k+ URLs to catch zero-day phishing with 99.58% accuracy.
2. **Multi-layered:** Combines AI with SSL, DNS, WHOIS, and Email header verification for defense-in-depth.
3. **Full-Stack & Production-Ready:** Backed by high-speed FastAPI backend, SQLite database, and responsive Next.js dark-mode UI.

7 Live Demonstration Script (Step-by-Step for Viva)

EXAMINER DEMO GUIDE

When the examiner asks you to demonstrate the project, follow this exact 4-step sequence:

1. **Show the Dashboard (Overview):**
   Open `http://localhost:3000`. Point out the live stats cards: Total Scans, Critical Threats, Safe Scans, and the Recent Scan Activity log. Explain that this gives a SOC (Security Operations Center) analyst immediate threat visibility.
2. **Demo 1: Scan a Legitimate URL:**
   Click on *URL Scanner* in the sidebar. Click the `https://google.com` example button and press *Scan Now*.
   Show the result: **Threat Score: 0/100 (LOW RISK - SAFE)**. Point out that the ML model gave 0.0% phishing probability, SSL certificate is valid, and WHOIS domain age is over 25 years old.
3. **Demo 2: Scan a Malicious / Suspicious URL:**
   Enter a typical phishing simulation: `http://paypal-security-update.com.verify-login.xyz/account`.
   Click *Scan Now*. Show the result: **Threat Score: 85+/100 (CRITICAL RISK - PHISHING DETECTED)**.
   Show the examiner the red badge breakdown: Brand impersonation keywords detected, suspicious TLD (`.xyz`), multiple subdomains, and lack of valid SSL.
4. **Show the Email Scanner:**
   Click on *Email Scanner*. Paste a simulated phishing email with urgent subject lines and fake links. Show how the engine parses the header, flags missing SPF/DKIM, and detects embedded attack links.
5. **Show the Model Metrics View:**
   Click on *Model Health* in the sidebar to prove to the examiner that the model has real metrics (99.58% Accuracy, 235k dataset size, ROC-AUC curve).